Question 1: Generative Adversarial Networks (GANs)

Q1(a) Explain the fundamental architecture of a GAN, describing the roles of the generator and discriminator networks. How do these two components interact during training, and what is the theoretical goal of this adversarial process?

Introduction

A **Generative Adversarial Network (GAN)** is a generative deep learning model introduced by Ian Goodfellow and colleagues in 2014. Unlike traditional machine learning models that primarily focus on classification or prediction, GANs aim to learn the underlying distribution of training data and generate new samples that resemble real examples.

GANs use an adversarial learning framework consisting of two neural networks: a **Generator** and a **Discriminator**. These networks have competing objectives and improve through repeated interaction during training.

1. Generator Network

The generator is responsible for producing synthetic data that resembles real training samples.

It begins with a randomly sampled latent vector, typically drawn from a normal or uniform distribution. The generator transforms this vector through several neural network layers to produce an output, such as an image.

Initially, the generator produces unrealistic or noisy outputs because its parameters have not yet learned meaningful characteristics of the dataset.

As training progresses, the generator learns patterns such as shapes, textures, edges, and structural relationships from the feedback it receives through the discriminator.

Example: When a GAN is trained on FashionMNIST, the generator initially produces random-looking grayscale images. After sufficient training, it may learn to generate recognizable shirts, shoes, trousers, and bags.

The generator's primary objective is to produce synthetic samples that the discriminator classifies as real.

2. Discriminator Network

The discriminator acts as a binary classification network that evaluates whether a given sample originates from the real training dataset or was generated artificially.

During training, it receives two types of inputs:

- **Real samples:** Original images from the training dataset.
- **Fake samples:** Synthetic images created by the generator.

The discriminator processes these inputs and produces a score representing how likely each sample is to be real.

By learning to distinguish authentic samples from generated ones, the discriminator develops an understanding of important characteristics of the real data distribution.

Example: For FashionMNIST, the discriminator may learn that realistic clothing images have recognizable silhouettes, appropriate pixel distributions, and consistent visual structures. It can use these characteristics to identify unrealistic generated images.

Its objective is to correctly distinguish real samples from fake samples.

3. Interaction Between Generator and Discriminator

The generator and discriminator are trained in an alternating adversarial process.

Step 1: Generate synthetic data

The generator receives random noise and transforms it into artificial samples. Initially, these outputs may contain distorted or unrecognizable features.

Step 2: Train the discriminator

The discriminator receives a batch containing real training images and generated images. It learns to assign higher realness scores to authentic images and lower scores to synthetic images.

Step 3: Train the generator

The generator produces another batch of synthetic images, which are passed through the discriminator.

The discriminator's feedback provides a training signal that allows the generator to update its parameters so its outputs are more likely to be classified as real.

During this update, the discriminator's parameters are held fixed, but its computations still provide gradients to the generator.

Step 4: Repeat adversarial training

Both networks continue improving through alternating optimization.

As the generator becomes better at producing realistic samples, the discriminator must learn increasingly subtle differences between real and synthetic data.

This competition encourages the generator to capture more complex features of the training distribution.

4. Theoretical Goal of GAN Training

The theoretical goal is for the generator to learn a probability distribution that closely matches the distribution of the real training data.

Ideally, generated samples should be statistically indistinguishable from real samples.

Under ideal assumptions, the original GAN framework has an equilibrium where the generated distribution matches the real distribution. At this point, an optimal discriminator cannot reliably distinguish real samples from generated samples and assigns approximately equal probability to both possibilities.

However, reaching this equilibrium is difficult in practice because GAN training involves two competing neural networks and can experience instability, vanishing gradients, or mode collapse.

Conclusion

GANs provide a powerful framework for generating realistic synthetic data through adversarial learning. The generator creates artificial samples, while the discriminator evaluates their authenticity. Their repeated competition enables the generator to progressively approximate the real data distribution.

Q1(b) Define mode collapse and explain why it occurs. Discuss at least two techniques proposed to mitigate this problem, explaining how each approach stabilizes training.

Introduction

One of the major challenges in training Generative Adversarial Networks is **mode collapse**.

Mode collapse occurs when a generator produces only a limited variety of outputs, even though the original dataset contains many different types of examples.

A mode represents a region or pattern of high probability within a data distribution. Ideally, a GAN should learn all significant modes of the training dataset rather than focusing on only a few.

1. What Is Mode Collapse?

In mode collapse, the generator repeatedly produces similar or nearly identical samples for different random noise inputs.

Although these generated samples may appear realistic individually, they fail to represent the diversity of the original dataset.

Example: FashionMNIST

FashionMNIST contains ten clothing categories, including shirts, trousers, coats, bags, and footwear.

A properly trained GAN should generate examples across the different categories and produce meaningful variation within each category.

However, a generator experiencing mode collapse might repeatedly generate sneakers while producing very few examples of the other clothing categories.

Another form of mode collapse occurs when the generator produces different images that share almost identical shapes or textures.

Therefore, a GAN can produce visually realistic outputs while still failing to learn the complete training distribution.

2. Why Does Mode Collapse Occur?

Reason 1: Generator exploits discriminator weaknesses

The generator may discover certain outputs that consistently fool the discriminator.

Instead of learning diverse representations of the dataset, it concentrates on these successful outputs because they provide favorable training feedback.

Reason 2: Unstable adversarial optimization

GAN training involves two neural networks whose objectives change as the other network improves.

If the generator or discriminator becomes significantly stronger than the other, the training process may become unstable.

This instability can cause the generator to focus on a narrow region of the data distribution.

Reason 3: Lack of explicit diversity constraints

The original GAN objective primarily encourages generated samples to appear realistic.

It does not directly require different noise vectors to produce sufficiently diverse outputs.

Consequently, the generator may learn to map many different latent inputs to similar outputs.

Reason 4: Oscillating training behavior

The generator may shift between producing different types of samples without successfully learning all modes simultaneously.

For example, it may first generate mostly shoes, later shift toward bags, and then return to generating shoes.

This prevents stable coverage of the full data distribution.

3. Technique 1: Minibatch Discrimination

Minibatch discrimination is a technique that allows the discriminator to examine relationships among multiple samples in a training batch.

In a conventional GAN, the discriminator generally evaluates each input independently.

As a result, it may identify whether an individual image looks realistic without recognizing that many generated images are nearly identical.

How Minibatch Discrimination Works

1. The discriminator extracts features from samples in a batch.
2. Additional computations compare the feature representations of different samples.
3. The discriminator uses information about similarity and diversity when evaluating generated examples.
4. If the generator repeatedly produces nearly identical samples, the discriminator can learn to identify this pattern.

How It Mitigates Mode Collapse

Because the discriminator can detect excessive similarity between generated samples, the generator receives feedback that encourages greater output diversity.

Instead of producing one highly realistic image repeatedly, the generator benefits from generating different realistic images.

Example: If a generator repeatedly produces the same type of sneaker, minibatch discrimination helps the discriminator detect the repeated pattern and penalize this behavior.

Advantages

- Encourages diversity in generated outputs.
- Helps the discriminator identify repeated samples.
- Addresses mode collapse by introducing batch-level information.

Limitation: It introduces additional computations and does not guarantee that every mode of the training distribution will be learned.

4. Technique 2: Wasserstein GAN with Gradient Penalty (WGAN-GP)

Wasserstein GAN with Gradient Penalty is an improved GAN training framework designed to address instability in adversarial optimization.

Traditional GAN training can suffer from weak or uninformative gradients, particularly when the real and generated data distributions have little overlap.

WGAN introduces a Wasserstein-based objective that provides a different measure of discrepancy between these distributions.

Instead of a conventional probability-based discriminator, WGAN uses a **critic** that assigns scores to samples.

WGAN-GP further improves this framework by adding a gradient penalty to encourage the critic to satisfy the required Lipschitz constraint.

How WGAN-GP Works

1. The generator produces synthetic samples from random noise.
2. The critic assigns scores to real and generated samples.
3. The critic is trained to distinguish the distributions through its scoring function.
4. A gradient penalty regularizes the critic and discourages excessively large gradient norms.
5. The generator receives feedback from the critic and updates its parameters to improve the generated distribution.

How It Mitigates Mode Collapse

The Wasserstein-based training objective can provide more useful gradients when the generated distribution differs substantially from the real distribution.

This helps the generator receive meaningful feedback instead of becoming trapped in poorly optimized regions.

The gradient penalty also improves the stability of critic training, making optimization more consistent.

As a result, WGAN-GP can reduce the tendency of the generator to concentrate on only a few modes.

Advantages

- Provides more informative training feedback in many difficult settings.
- Improves adversarial training stability.
- Avoids the weight-clipping approach used in the original WGAN.
- Can reduce mode collapse and improve sample quality.

Limitation: WGAN-GP does not completely eliminate mode collapse, and its performance still depends on architecture, optimization settings, and training data.

5. Comparison of Both Techniques

Feature	Minibatch Discrimination	WGAN-GP
Main focus	Sample diversity	Training stability
Approach	Compares samples within a batch	Uses Wasserstein-based critic training with gradient penalty
Primary benefit	Detects repetitive generated outputs	Provides more stable and informative gradients
Effect on mode collapse	Directly discourages repeated samples	Can indirectly reduce collapse through improved optimization
Limitation	Additional batch-level computation	Requires careful training configuration

Conclusion

Mode collapse reduces the diversity of generated samples and prevents GANs from representing the complete training distribution.

Minibatch discrimination addresses this problem by helping the discriminator detect repetitive outputs, while WGAN-GP improves optimization stability and gradient quality.

Together, these techniques illustrate two complementary strategies for improving GAN training: encouraging diversity and stabilizing the adversarial learning process.

Q1(c) Describe three significant variants or extensions of the original GAN architecture. For each variant, explain the limitation it addresses and the innovations it introduces.

Introduction

The original GAN architecture introduced a powerful adversarial learning framework, but it also had several limitations.

These included difficulties in generating high-quality images, limited control over generated outputs, and the need for paired training data in certain image transformation tasks.

Researchers developed multiple GAN variants to address these challenges.

Three significant extensions are **Deep Convolutional GAN (DCGAN), Conditional GAN (cGAN), and CycleGAN**.

1. Deep Convolutional GAN (DCGAN)

Overview

Deep Convolutional GAN, introduced by Radford and colleagues, extends the original GAN framework by incorporating convolutional neural network architectures.

Convolutional networks are particularly suitable for image processing because they can learn spatial relationships among neighboring pixels.

Limitation of the Original GAN

Simple fully connected GAN architectures do not explicitly exploit the spatial structure of images.

This can make learning complex visual patterns inefficient and may limit the quality of generated images.

Fully connected architectures also tend to require many parameters when processing high-dimensional images.

Key Innovations

A. Convolutional architecture

DCGAN uses convolutional layers in the discriminator to extract image features such as edges, shapes, and textures.

The generator typically uses transposed convolutional layers to transform low-dimensional latent representations into images.

B. Batch normalization

Batch normalization is used in much of the architecture to help stabilize training and improve gradient propagation.

C. Specialized activation functions

DCGAN commonly uses ReLU activations in the generator's hidden layers, Tanh in its output layer, and LeakyReLU in the discriminator.

D. Hierarchical feature learning

The model learns visual representations at multiple levels, from simple edges and textures to more complex object structures.

Example

When trained on FashionMNIST, a DCGAN can learn spatial features such as clothing outlines, shoe shapes, and bag structures.

Its convolutional design helps it capture image-specific patterns more effectively than a basic fully connected implementation.

Advantages

- Better suited to image generation.
- Learns meaningful spatial features.
- Provides architectural practices that can improve training stability.
- Supports more effective visual representation learning.

Limitation

DCGAN can still experience mode collapse and adversarial training instability, especially on complex datasets.

2. Conditional Generative Adversarial Network (cGAN)

Overview

A Conditional GAN extends the original GAN framework by allowing generated outputs to be controlled using additional information.

This information may include class labels, attributes, text descriptions, or other conditioning variables.

Limitation of the Original GAN

In an unconditional GAN, the generator receives random noise without an explicit instruction about which type of sample to produce.

For example, a standard GAN trained on FashionMNIST may generate a shoe, shirt, or bag, but the user cannot directly specify the desired category.

This limits the model's usefulness in applications requiring controlled generation.

Key Innovations

A. Conditioning information

The generator receives both random noise and a condition, such as a class label.

The discriminator also receives the condition and evaluates whether the sample is realistic and consistent with it.

B. Controlled generation

The generator learns to produce outputs that correspond to the specified condition.

C. Conditional adversarial learning

The discriminator provides feedback about whether generated samples match the requested characteristics as well as whether they appear realistic.

Example

Consider a cGAN trained on FashionMNIST.

If the user supplies the label "ankle boot," the generator learns to produce an ankle boot rather than an arbitrary clothing item.

Similarly, supplying the label "bag" encourages the model to generate a bag.

Random noise can still introduce variation within the requested category.

Applications

- Class-specific image generation.
- Text-conditioned image synthesis.
- Image-to-image translation.
- Data augmentation for underrepresented categories.

Advantages

- Provides control over generated outputs.
- Allows targeted generation for particular classes or attributes.
- Makes GANs more useful for task-specific applications.

Limitation

The quality of conditional generation depends on the training data and the model's ability to learn the relationship between conditions and outputs. Conditioning does not automatically prevent mode collapse.

3. Cycle-Consistent Generative Adversarial Network (CycleGAN)

Overview

CycleGAN is designed for image-to-image translation between two different visual domains without requiring paired examples.

It is particularly useful when corresponding images from the two domains are difficult to obtain.

Limitation of Earlier Image Translation Methods

Many supervised image-to-image translation approaches require paired training examples.

For example, translating photographs into paintings may require matching pairs where each photograph has a corresponding painting of the same scene.

Collecting such paired datasets can be expensive or impractical.

CycleGAN addresses this problem by learning translations from unpaired collections of images.

Key Innovations

A. Two generators

CycleGAN uses two generators:

- The first translates images from Domain A into Domain B.
- The second translates images from Domain B back into Domain A.

B. Two discriminators

Each domain has its own discriminator.

One discriminator evaluates whether translated images resemble real examples from Domain A, while the other evaluates whether translated images resemble real examples from Domain B.

C. Cycle-consistency loss

CycleGAN introduces a cycle-consistency objective.

When an image is translated into another domain and then translated back, the reconstructed image should resemble the original.

This discourages arbitrary transformations that discard important information.

D. Unpaired training

The model can learn from two separate image datasets without requiring one-to-one correspondence between images.

Example

Consider two datasets:

- Domain A: Photographs of horses.
- Domain B: Photographs of zebras.

CycleGAN learns to transform horse images into zebra-like images while attempting to preserve important structural features, such as the animal's shape and pose.

It can also learn the reverse transformation from zebras to horses.

The training process does not require paired photographs of the same animal in both forms.

Applications

- Artistic style transfer.
- Seasonal image transformations.
- Photograph-to-painting translation.
- Visual domain adaptation.
- Certain medical image translation and enhancement research tasks.

Advantages

- Does not require paired training images.
- Supports translation between different visual domains.
- Uses cycle consistency to encourage preservation of important image information.
- Reduces dependence on expensive paired datasets.

Limitations

CycleGAN may alter important details, introduce visual artifacts, or fail to preserve semantic information in complex transformations.

In medical applications, this is particularly important because generated images may introduce or remove clinically meaningful features. Therefore, outputs require careful validation.

4. Comparison of the Three GAN Variants

Feature	DCGAN	Conditional GAN	CycleGAN

Primary purpose	Improve image generation architecture	Control generated outputs	Translate between image domains
Original limitation addressed	Limited use of spatial image structure	No explicit control over output category	Dependence on paired image data
Key innovation	Convolutional networks	Conditioning variables	Two-way translation and cycle consistency
Example	Generate clothing images	Generate a specified clothing class	Convert horses into zebras
Training data	Images	Images with conditions	Two unpaired image datasets
Main advantage	Learns spatial visual features	Enables targeted generation	Enables unpaired image translation

Conclusion

GAN variants extend the original adversarial learning framework to address different practical limitations.

DCGAN improves image generation by using convolutional architectures that learn spatial features.

Conditional GAN introduces conditioning information, allowing users to control the characteristics of generated samples.

CycleGAN enables image-to-image translation between two domains without requiring paired training examples.

These developments demonstrate how the original GAN framework has evolved into a flexible family of generative models supporting image synthesis, controlled generation, and visual transformation.